

iOSDC Japan 2026 ・ Regular Talk ・ 40 min

映像変換サーバーなしで iPhone端末内で HLSを生成してライブ配信

Hikaru Sato @SatoHikaruDev

playlist.m3u8

init.mp4

000001.m4s

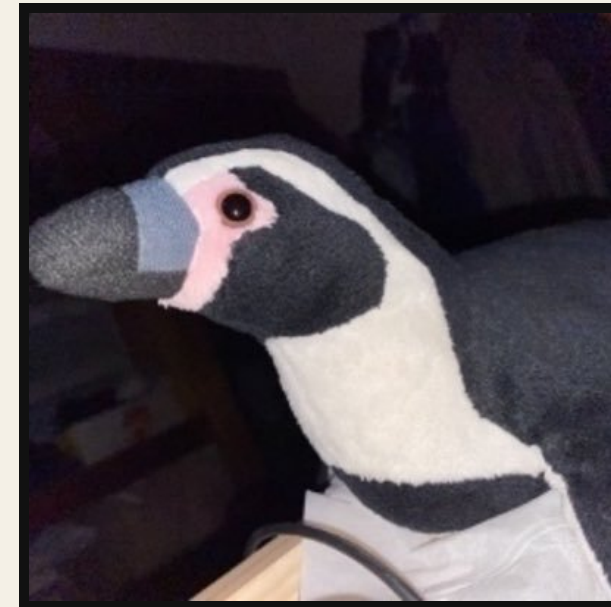
— ABOUT ME

Hikaru Sato

@SatoHikaruDev

主に iOS / Android / Ruby on Rails のアプリ開発をしています

[MomentNow](#) という「今この瞬間」の動画をHLS
で配信し、
URLで共有できる iOSアプリ を個人開発してます



今回は MomentNow での端末内HLS生成の仕組みや実装を分解します

話すこと

WHY なぜ端末内HLS生成をやろうと思ったのか

01 HLSがライブになる仕組み

02 端末内でHLSを生成

03 動画断片の生成

04 playlistの更新

05 配信の完了

— WHY

撮影とuploadを重ね、共有までの待ち時間を短くする



模式図：撮影中に送信を進め、停止後の大きなuploadを減らす

同じplaylist URLで共有。停止後も最後の送信と終了処理は残る

macOS向けAppleサンプルが とても参考になった

2025.03

AIで最初の試作

HLSとしての
実用まで至らず



Apple公式

生成手順を確認

Writer設定
時刻補正
Dataの受け取り



iPhone local

HLS生成に成功

カメラ・マイク
短い動画を生成



MomentNow

ライブ配信へ発展

撮影中からupload
playlistを更新

端末内生成で変換サーバーを減らせると分かり、MomentNowの開発へ進んだ

01

HLSで「ライブ」になる仕組み

短い動画ファイル + 更新され続けるplaylist

playlistが、取得するファイルと再生順を示す

```
#EXT-X-MAP:URI="init.mp4"
```



init.mp4

再生設定

```
#EXTINF:2.000,  
seg/000001.m4s
```



000001.m4s

最初の約2秒

```
#EXTINF:2.000,  
seg/000002.m4s
```



000002.m4s

次の約2秒

Media Playlistの抜粋。ヘッダーなどは省略

Playerはplaylistを読み、書かれたURIからHTTPで取得する

— THREE FILE TYPES

HLSライブ配信を構成する、3種類のファイル

M3U8	MP4	M4S
playlist.m3u8 更新される再生順	init.mp4 再生を始めるための設定	000001.m4s 2秒前後の映像と音声

init.mp4（再生準備） + m4s（約2秒） + playlist（順番） = 最初の再生

fMP4は、再生設定と映像・音声の断片を分ける

init.mp4 · 最初に1回

再生設定

圧縮方式や

映像・音声の構成

+

m4s · 約2秒ごと

映像 + 音声

断片 1 断片 2 断片 3 ...

↓

playlistが両方のURIと順番を結ぶ

Playerが組み合わせて再生する

init単体・m4s単体を、完成した録画ファイルとして扱わない

最初は、playlistを読んで 再生に必要なファイルを取得

Player
playlistをGET



サーバーのplaylist
init + segment 1



① 再生設定を取得
GET init.mp4



② 映像・音声を取得
GET 000001.m4s

再生可能なファイルが揃う。再生開始のタイミングはPlayerにも依存する

同じURLを再取得すると、 新しいsegmentが見つかる

前回のplaylist

init + segment 1

すでに取得した内容



今回のplaylist

init + segment 1 · 2

末尾にsegment 2が追加



Playerが新しいURIを発見

GET 000002.m4s



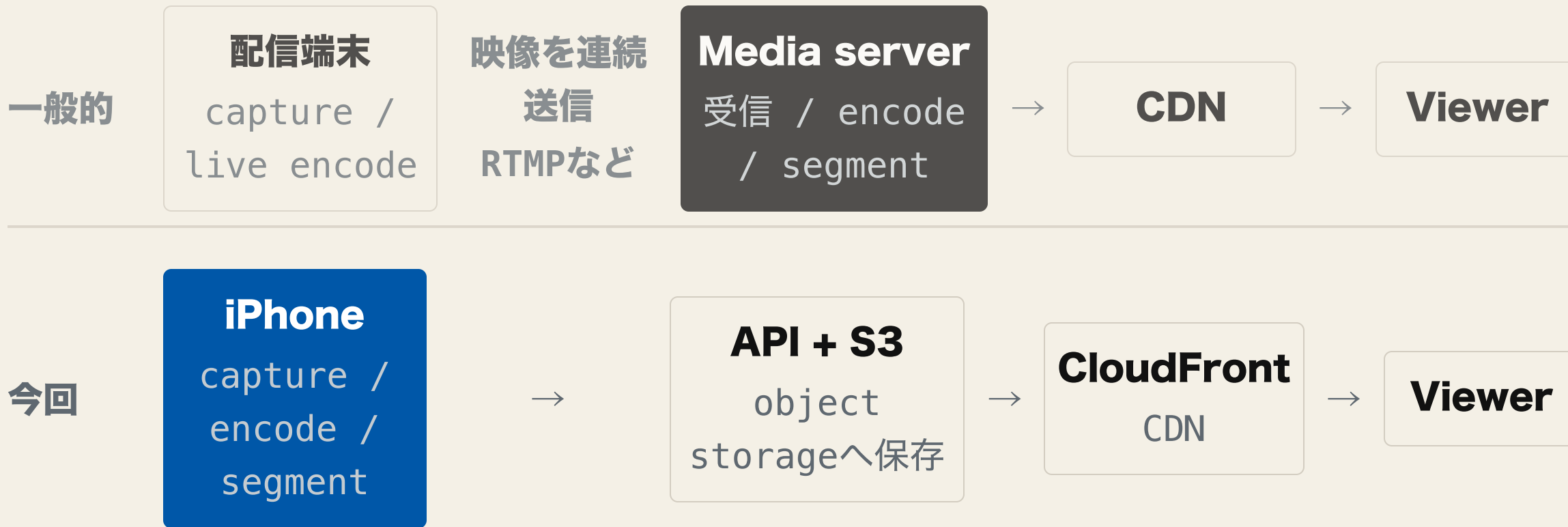
映像と音声を追加

続きへ再生が進む

ライブ中は繰り返す。終了時のENDLISTで「これ以上増えない」と伝える

— FROM HLS TO THIS TALK

HLSを作る場所を、サーバーからiPhoneへ



今回なくすのは、映像を受信し続けて変換するサーバー

— THIS TALK'S IMPLEMENTATION

iPhoneが生成したHLSを 撮影中からS3へ公開する

映像と音声を受け
取る

Camera + Mic



約2秒に分ける

AVAssetWriter



保存先へ置く

HTTP PUT



再生順を公開する

playlist更新

撮影・保存・接続に必要な権限

権限	このサンプルでの用途	許可するタイミング
カメラ	配信映像の撮影	プレビュー開始時
マイク	配信音声の録音	プレビュー開始時
写真へ追加	完成したMP4の保存	配信開始前
ローカル ネットワーク	同じLANのMacへ接続	最初のLAN接続時

Info.plistに用途を記載。写真は「追加のみ」を要求する

— LIVE DEMO · PUBLIC SAMPLE

公開サンプルを動かす：生成・保存・再生

iPhone

Camera + Mic

2つのWriterで生成



Mac HTTP Server

HLS一式を保存

HTTP PUTで受信
映像変換なし



Viewer

追従再生

同じplaylistをGET

iPhone内の別経路・停止後

完成MP4を写真へ保存して再生

フルHD・HEVC。MP4はMacへ送信しない

Safariでデモ。同じplaylist URLをAVPlayerへ渡した再生確認にも触れる

Server は「PUTされたファイルを残すだけ」

Mac

```
$ python3 server/server.py
```

Viewer: <http://localhost:8080>

iPhone server URL:

<http://192.168.x.x:8080>

PUT init.mp4 / m4s / m3u8

DISK server/data/streams/...

GET 同じファイルをViewerへ
返す

「配信開始」で、iPhoneがHLS生成を始める

● 配信開始

→

Camera + Mic
video frame /
audio block
(CMSampleBuffer)

→

AVAssetWriter
圧縮してHLS用Dataへ
分割

elapsed 0.0 → segment 0 →

最初に届くのは、1回だけのinit.mp4

$t \approx 0$

initialization
callback



init.mp4

再生を始めるための設定

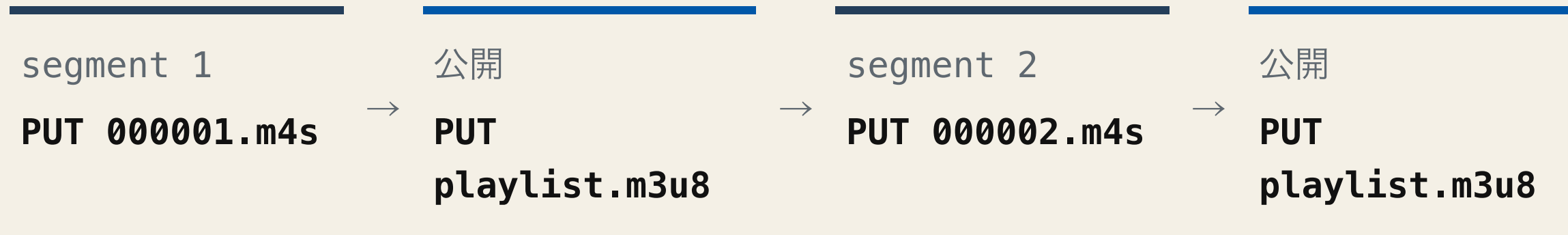


PUT

/streams/{id}/init.mp4

media segmentより先に、再生の前提を保存する

以後は約2秒ごとに、2つのPUTが続く



playlistが指すのは、保存に成功したsegmentだけ

Viewerは最新streamを見つけ、playlistを追いかける

1秒ごと

GET /api/streams



最新を選択

playlistURL



LIVE

Safari / AVPlayer /
hls.js

配信中 segmentが増える 停止後 写真へMP4を自動保存

配信後は、MacとiPhoneに別々の動画が残る

Mac · HLSの実出力例

streamディレクトリ

init.mp4

playlist.m3u8

seg/000001.m4s

...

seg/000018.m4s

iPhone · 停止後に写真保存

1本のMP4

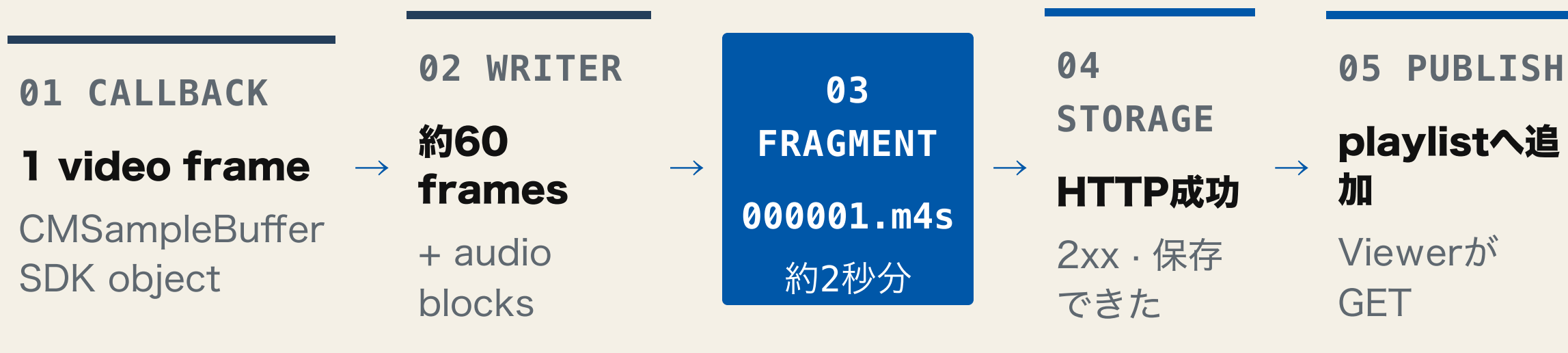
1080 × 1920

HEVC / AAC

保存用Writerで別途生成

Macはplaylistから再生。iPhoneは写真アプリで保存動画を再生

1 frameは、約2秒のsegmentへ まとめてから届ける



frame単体は送らない。約2秒分を保存してから、playlistで見えるようにする

02

Camera / Micから、 fMP4 Dataを取り出す

Camera / Mic → 1 frameずつ受け取る → Writer → HLS用Data

Writerが作るDataを、アプリが保存・公開する

AVFoundation

AVAssetWriter

映像・音声を圧縮
約2秒に分割



delegateの出力

init / media Data

まだファイル名はない



アプリ側

Publisher

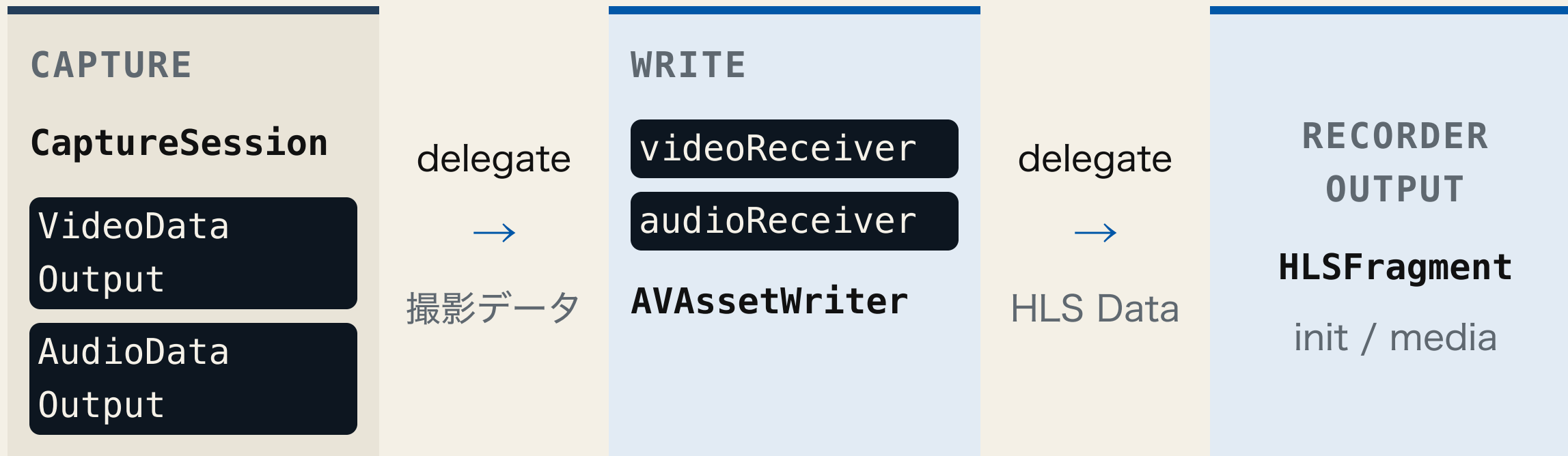
名前を決めて保存
成功後にplaylist公開

Writerの担当：HLS用のData生成

アプリの担当：保存先・HTTP・playlist

AVAssetWriterは、playlistを生成しない

HLS生成は、6つのSDK objectを接続する



撮影データを、HLSとして保存できるDataへ変える

CaptureSessionが、 Camera / MicをDataOutputへつなぐ

Back camera

AVCaptureDeviceInput

Microphone

AVCaptureDeviceInput

AVCaptureSession

VideoDataOutput

video CMSampleBuffer

AudioDataOutput

audio CMSampleBuffer

完成した録画ファイルではなく、撮影中のframe / audio blockを受け取る

1つのCaptureSessionを、3つの用途で使う

Camera + Mic
CaptureSession



PreviewLayer

画面表示：Cameraの映像を表示



HLS用Writer

ライブ配信：短いfMP4 Dataを生成



保存用Writer

端末保存：1本のMP4を生成

Sample · MomentNow共通。撮影データを2つのWriterへ入力する

— TWO WRITERS · TWO DESTINATIONS

配信はH.264、端末保存はフルHD・HEVC

Sample · HLS用Writer

H.264 / AAC

720 × 1280

映像 1.5 Mbps



init + m4s

撮影中からMacへPUT

Sample · 保存用Writer

HEVC / AAC

1080 × 1920

映像 5 Mbps



完成MP4

停止後に写真へ自動保存

MomentNowは元動画upload + 設定に応じた写真保存も行う

映像と音声のcallbackを、 1本のqueueで順に処理

VideoDataOutput

映像1 frameずつ

AudioDataOutput

短い音声blockずつ



共通のserial writingQueue 時間 →

video 1



audio 1



video 2



audio 2

各callback内で、Writer開始・時刻処理・入力を順に行う

Writerの状態を同時に更新しない。映像と音声の時刻はPTSで扱う

2つのDataOutputに、同じqueueを指定する

```
1 videoOutput.setSampleBufferDelegate(  
2     self, queue: writingQueue)  
3 audioOutput.setSampleBufferDelegate(  
4     self, queue: writingQueue)
```

delegate

self

captureOutputで受信

callbackの実行先

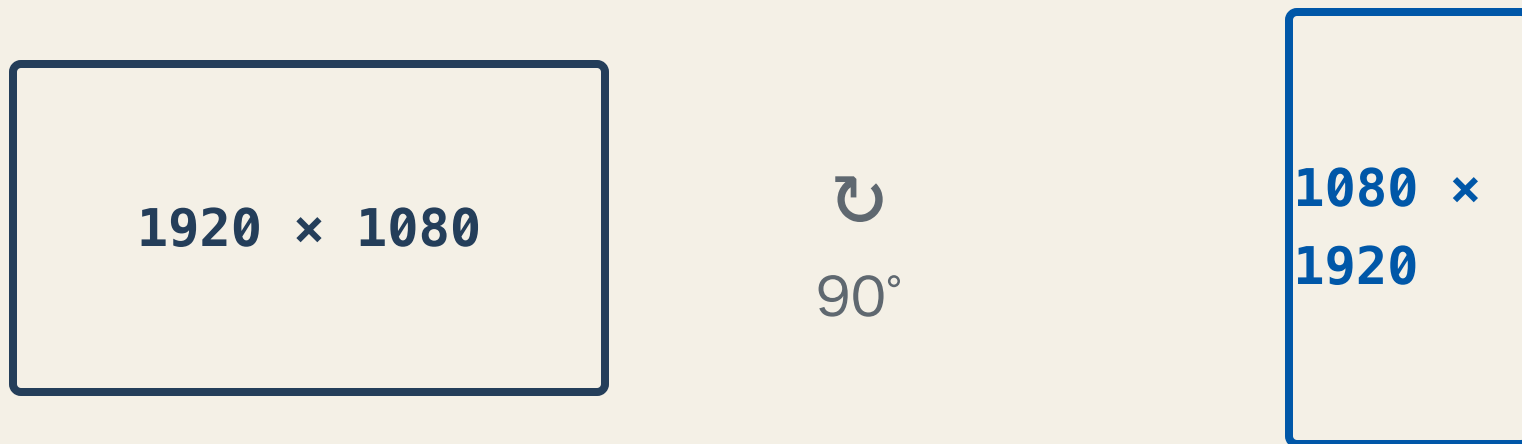
→ **writingQueue**

同じserial queue

設定は準備時に1回。撮影中は同じcallbackが繰り返し呼ばれる

— ORIENTATION

縦向きは、video connectionへ90度を指定



```
1 if let connection = videoOutput.connection(with: .video),
2   connection.isVideoRotationAngleSupported(90) {
3   connection.videoRotationAngle = 90
4 }
```

配信開始前に、上り回線に合う単一品質を選ぶ

NWPath

回線の状態

API応答

準備の待ち時間

test upload

上り速度



MomentNow : 開始前に1つ選択

Low · 480 × 854

0.9 Mbps

Medium · 720 × 1280

1.5 Mbps

High · 720 × 1280

2.5 Mbps

Sampleは720 × 1280 · 1.5 Mbps固定

選んだ品質で配信する。配信途中の自動画質切替（ABR）は行わない

未圧縮映像を、WriterでH.264へ圧縮

Capture

未圧縮映像

video frame

High H.264の圧縮profile名

append

Writer

H.264 High

圧縮された映像

```
1 videoOutput.videoSettings = [  
2     kCVPixelBufferPixelFormatTypeKey as String:  
3     kCVPixelFormatType_420YpCbCr8BiPlanarFullRange  
4 ]
```

サンプルは1.5 Mbps・個人アプリは上り回線に合わせて0.9-2.5 Mbps

— VIDEO SETTINGS

配信用videoフォーマットは、 互換性と上り帯域を優先する

KEY	VALUE	INTENT
AVVideoCodecKey	H.264	広い再生互換性
Width / Height	480p / 720p	quality別に選択
AverageBitRate	0.9–2.5 Mbps	上り回線へ追従
ProfileLevel	High Auto	encoder profile
FrameReordering	false	decode順を単純化

AudioはAAC mono、品質別に48-96kbps

FORMAT	CHANNEL	SAMPLE RATE	BITRATE
AAC	mono	44.1 kHz	48-96 kbps

```
1  [  
2    AVFormatIDKey: kAudioFormatMPEG4AAC,  
3    AVNumberOfChannelsKey: 1,  
4    AVSampleRateKey: 44_100,  
5    AVEncoderBitRateKey: config.audioBitrate  
6  ]
```

URLを持たないWriterから、Dataを受け取る

HLS向け設定

AVAssetWriter



delegate

init / media Data

```
1 let writer = AVAssetWriter(contentType: .mpeg4Movie)
2 writer.outputFileTypeProfile = .mpeg4AppleHLS
3 writer.preferredOutputSegmentInterval = .init(
4     seconds: 2, preferredTimescale: 600)
5 writer.initialSegmentStartTime = startTimeOffset
6 writer.delegate = self
```

ファイル保存用のoutputURLは指定しない

Inputは圧縮設定、Receiverはbufferの入口

① 準備時に設定

Input

mediaType

outputSettings



② Writerへ接続

Receiver

入力窓口を取得して保持



③ 生成を担当

Writer

映像・音声を圧縮

fMP4 Dataを出力

撮影中の繰り返し

CMSampleBufferをReceiverへ入力する

videoReceiver / audioReceiverを使い分ける

Inputは設定。撮影データを書き込む窓口はReceiver

InputをWriterへ接続し、Receiverを保持する

```
1 let videoInput = AVAssetWriterInput(  
2     mediaType: .video,  
3     outputSettings: videoSettings()  
4 )  
5 self.videoReceiver = writer.inputReceiver(for: videoInput)
```

setup内で作成
videoInput



以後のcallbackで使用
videoReceiver

AudioもmediaTypeとoutputSettingsを替え、同じ手順で接続する

小さなbufferをまとめ、 約2秒のsegmentにする

撮影中に届く入力の例



init Dataは最初に1回。media Dataは撮影中に繰り返し届く

Writerへ渡す前に、2つのguardで確認する

captureOutput

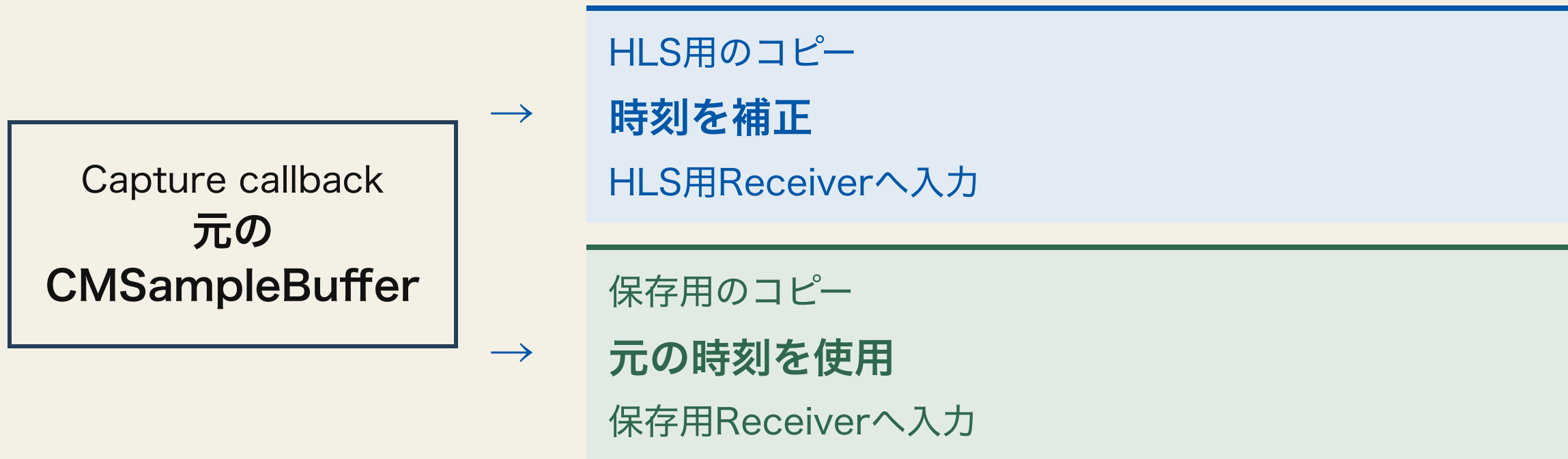
配信中か
isWriting

Dataはreadyか
利用可能か確認

Writer開始
→ append

```
1 guard isWriting else { return }  
2 guard CMSampleBufferDataIsReady(sampleBuffer) else { return }
```

同じ撮影データをコピーし、2つのWriterへ



HLS側が受け取れなくても、保存用Writerへの入力は続ける

HLS側のReceiverへ、待たずに入力を試す

```
1 let readySampleBuffer = CMReadySampleBuffer(  
2     unsafeBuffer: transferableSampleBuffer)  
3 let didAppend = try receiver.appendImmediately(  
4     readySampleBuffer)  
5 guard didAppend else { return }
```

true

入力できた

false

このbufferをdrop

throw

HLS側が失敗

保存用Writerへの入力は別に実行する

WriterのDataを、Publisherが保存する

`.initialization` → init Data

→ `init.mp4`

`.separable` → media Data

→ `seg/000001.m4s`

```
1 switch segmentType {
2 case .initialization:
3     continuation.yield(.initialization(segmentData))
4 case .separable:
5     segmentIndex += 1
6     continuation.yield(.media(sequence: segmentIndex,
7         data: segmentData, duration: duration))
8 }
```

Recorderの出力はData。HTTPHLSClientが保存先のpathを組み立てる

HLS生成の準備：SDK objectを接続する

01

Capture

Camera
/ Micを
Sessionへ追加



02

DataOutput

delegateと
serial queue



03

Writer

HLS profile
約2秒の設定



04

Receiver

Inputを接続し
窓口を保持

準備では、入力元・callbackの実行先・生成先をつなぐ

HLS生成の実行：開始・入力・受信・終了

05

開始

最初のVideoで
startSession



06

入力

時刻を補正
Receiverへ
append



07

Dataを受信

init：1回
media：反復



08

終了

finishWritingで
最後のData

05・06の時刻処理は、次の1枚で確認する

— WRITER TIMING

映像と音声の時刻を、同じ量だけ移動する

HLS側はApple公式サンプルの方針を採用

調整前

100.00

映像

100.02

音声

時間差 0.02秒

−90秒

→

Writerへ渡す時刻

10.00

映像

10.02

音声

時間差 0.02秒

検証結果：時刻調整を外すと、映像・音声が入らなかった

検証条件：Writerの開始位置は10秒のまま、時刻調整だけを外した場合

03

映像と音声を、 約2秒のfMP4へ分ける

Four HLS settings and segment boundaries

4つの設定が、出力形式と境界・受け口を決める

`outputFileTypeProfile`



Writerの出力形式
Apple HLS向けfMP4

`preferredOutputSegmentInterval`



segmentの間隔
約2秒ごとの境界

`initialSegmentStartTime`



最初の開始位置
時刻補正に合わせる

`delegate`



Dataの受け口
init / mediaを受信

約2秒付近のIDRを境界に、segmentを作る



模式例：2秒ぴったりになるとは限らない

実durationの反映：サンプルで実装済み

公開サンプル ・ 実装済み

実durationをEXTINFへ

reportが無効・未取得の場合だけ2秒

MomentNow ・ 現状

2秒固定で送信

改善案：同じ実durationの取り込みを適用

```
1 let duration = report?.trackReports
2   .first { $0.mediaType == .video }?
3   .duration.seconds
4 guard let duration, duration.isFinite, duration > 0 else {
5   return config.segmentSeconds
6 }
7 return duration
```

— KEYFRAME ALIGNMENT

IDRを用意すると、 segmentの先頭から再生できる

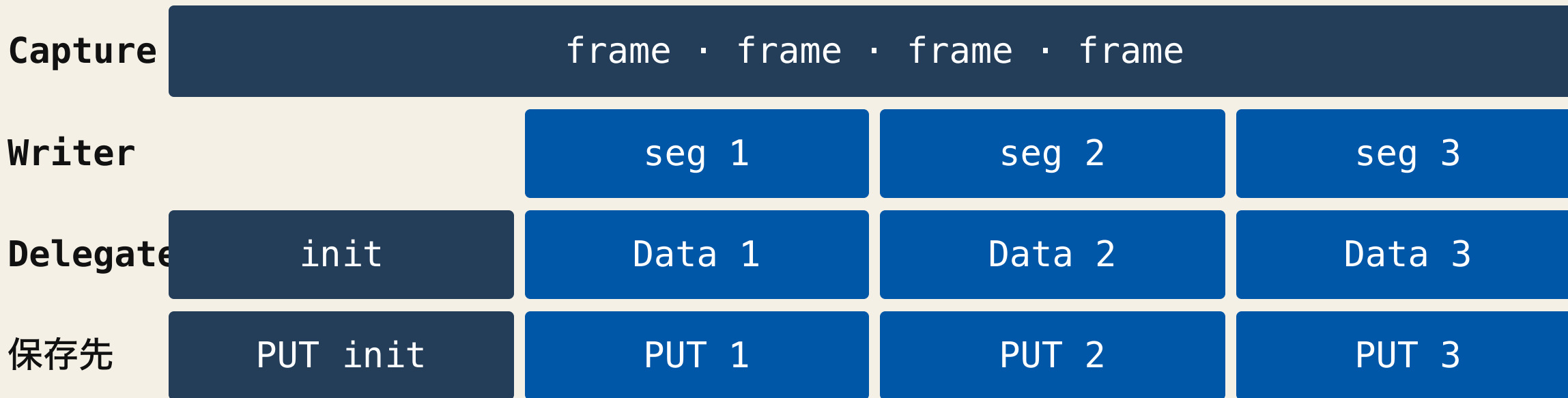
映像frameの依存関係（模式図）



1 AVVideoMaxKeyFrameIntervalDurationKey: `config.segmentSeconds`

Appleは約2秒ごとのIDRを推奨。segmentの希望間隔と合わせる

Captureを止めず、約2秒ごとにDataを出す



Writerのfinishを待たず、生成と保存を並行できる

04

保存できたsegmentだけを、 playlistへ公開する

Save bytes first, then make the segment visible to viewers

— SAME HLS · DIFFERENT DESTINATION

同じHLSを、MacまたはS3へ置く

公開サンプル	iPhone	HTTP PUT	Macのファイル保存	HTTP GET	ローカル Viewer
MomentNow	iPhone	署名付き URLへPUT	Amazon S3 object storage	CloudFront CDN	Web Viewer

init.mp4とm4sのDataは、どちらもiPhoneが生成する

書き込み中は、古い完成ファイルを公開し続ける

① PUTを受信中

一時ファイルへ書く

flush + fsyncまで完了させる

その間のViewer

古い完成版をGET

一時ファイルは参照しない



② os.replace

公開pathを一気に置換

書き切ったファイルへ切り替える

置換後のViewer

新しい完成版をGET

途中の内容を見せない

playlistの途中状態や、半分だけのm4sを公開しない

— PLAYABLE STATE

Dataの生成だけでは、 Viewerからは再生できない

端末内

Dataを生成

Writer delegateで受信

保存先

→ **init + m4sを保存**

HTTP成功を確認

公開

→ **playlistに掲載**

URIから取得可能になる



Viewer

playlistを読み、initとsegmentをGETして再生

保存と公開が揃うと再生可能になる。実際の開始時刻はPlayerにも依存する

PUT成功を確認してから、 候補のplaylistを採用する

アプリ内の現在値
segment 1



コピーして候補を作成
segment 1・2



サーバーへ送信
playlist PUT



成功を確認できた
現在値を1・2へ更新

成功を確認できない
現在値は1のまま

segment 2本体は保存済み。この図はplaylistとアプリ内状態の更新

候補を送信し、成功後にmanifestへ代入する

```
1 var nextManifest = manifest
2 nextManifest.addSegment(seq: seq, durationSec: durationSec)
3 try await client.putPlaylist(
4     streamId: streamId, text: nextManifest.text)
5 manifest = nextManifest
```

通信前

nextManifestだけ変更



awaitが成功した後

manifestを置き換える

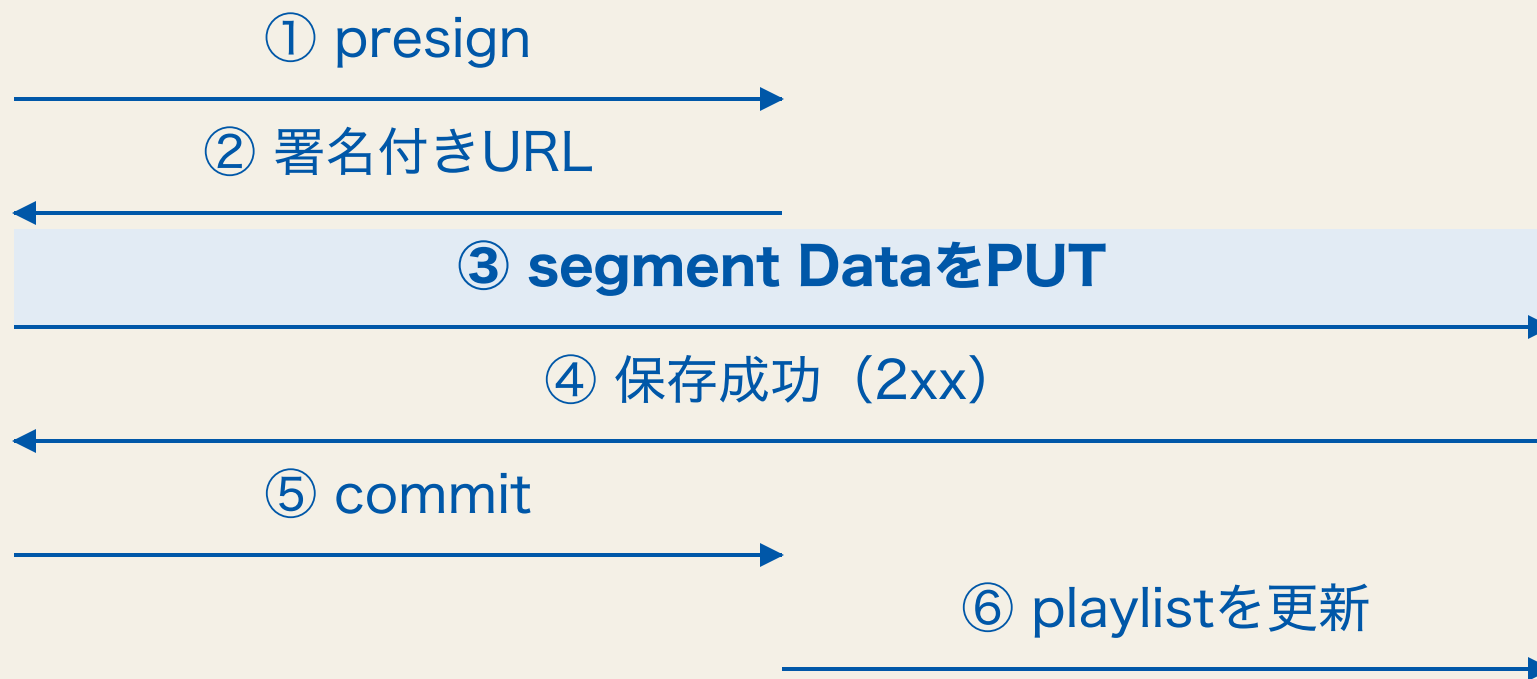
PUTがthrowした場合、最後の代入へ進まない

S3へDataをPUTし、APIでplaylistを公開する

iPhone

API

S3



APIは保存の許可と公開を制御。映像Dataを変換しない

S3へのPUTを、署名付きURLで許可する

DO NOT

AWS credentialを内包

漏えい範囲が広い



PRESIGNED URL

S3へ1ファイルだけPUT

保存先 / 有効期限を署名

iPhoneは発行されたURLへDataをPUTするだけ。AWS credentialは持たない

```
1 var request = URLRequest(url: presignedURL)
2 request.httpMethod = "PUT"
3 request.setValue(
4     contentType, forHTTPHeaderField: "Content-Type")
5 request.httpBody = data
```

— INIT ONCE · MEDIA REPEATS

mediaは、PUT後にcommitを追加

```
1 let url = try await api.presign(kind: kind, seq: seq)
2 try await api.put(to: url, data: data)
```

INITIALIZATION · 1回

```
kind: "init", seq: nil
```

PUT成功で完了

MEDIA · 約2秒ごと

```
kind: "segment", seq: seq
```

PUT成功後に追加

```
try await api.commit(seq: seq)
```

mediaだけは、PUT成功後にcommitしてplaylistへ載せる

— SAVE BEFORE PUBLISH

segmentの保存前に公開すると、Viewerは404になる

順序を逆にした場合

playlistへ先に掲載



ViewerがGET



本体がまだない
404

今回の順序

本体のPUT成功



playlistへ掲載



Viewerが取得可能

同じPUTを最大3回試し、 失敗したら公開を進めない

1回目が失敗

250 ms待つ



2回目が失敗

500 ms待つ



3回目

最後の試行



いずれかで成功

playlistのPUTへ進む

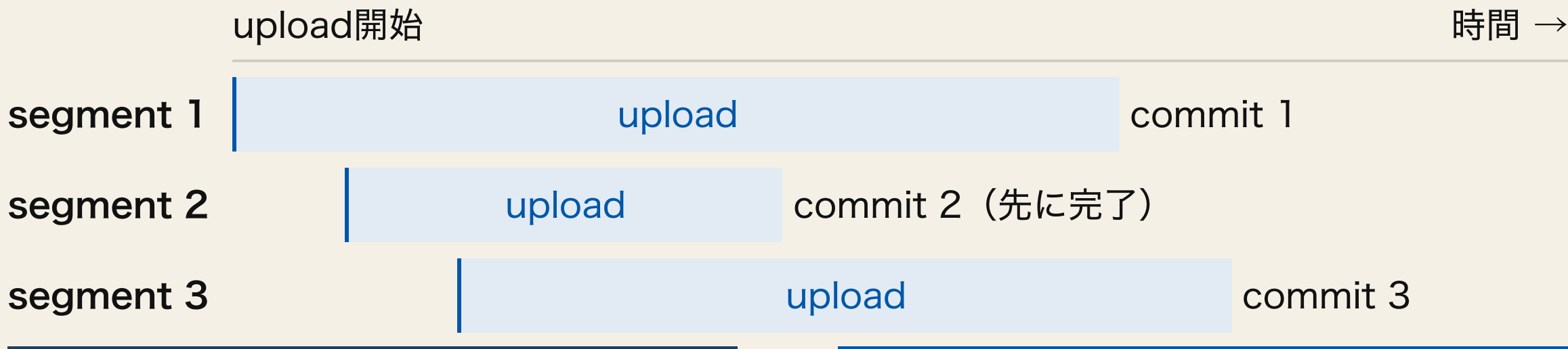
3回とも失敗

公開せずerrorを残す

初回を含めて最大3回。同じURLへ同じ内容を送る

segment本体のPUTが成功するまで、そのsegmentをplaylistへ載せない

uploadの完了順と、再生順は別に扱う



完了順の例

2 → 1 → 3

MomentNowの現状

→ sequence順へ整列

1 → 2 → 3

整列だけでは、途中の欠番を待てない

欠番があるとき、どこまで公開するか

segment 1・3が保存済み、segment 2はまだ届かない時点

1 保存済み

2 未到着

3 保存済み

MomentNow · 現状

1・3を番号順に掲載

欠番は待たない

1

3

改善案 · 未実装

連続した1だけを公開

3は2が届くまで保留

1

2待ち

3保留

番号順への整列と、連続した範囲だけの公開は別の制御

playlistは同じURLでも内容が増える

前回の内容

segment 1



サーバーの最新

segment 1・2



古い内容を使い続けると

segment 2を発見できない

今回のキャッシュ方針

no-store / no-cache

古いplaylistを使い続けない

変わるのはplaylistの内容。Playerは同じURLを繰り返し取得する

initとsegmentは内容を変えず、 キャッシュを再利用する

保存済み

000001.m4s

同じURLでは
内容を変えない



CDN / キャッシュ

完成Dataを保持

max-age=31536000



Viewer

同じDataを取得

再利用できる

次のsegment

000002.m4sは別URL

部分取得への対応

Range request / 206

更新されるplaylistと、変わらないメディアで方針を分ける

05

生成を止めず、 配信の完了まで待つ

AVFoundation callback → Swift Task → final playlist

callbackはすぐ戻り、HTTPは別のTaskで待つ

Writer delegate の値を writingQueue へ戻して受け渡す



受け渡しはAsyncThrowingStream。Taskが順番に取り出す

HTTP待ちでcallbackを止めない。ただし、送信待ちの上限は未設定

Writer側のdropと、生成後の送信待ち

生成前

CMSampleBuffer

frame / audio block



Writer

圧縮・分割



生成後

segment
Data

送信待ちの列



HTTP

送信

Writer側・実装済み

受け取れないbufferをdrop

映像のカクつき・音声の欠けを許容

送信側・改善案

滞留量の上限と停止判断

Sampleでは未実装

別の位置・別のデータに対する問題。送信待ちは現在、上限なし

— WRITER FINISH

HLS用Writerを、最後に渡した時刻で終了する

lastAdjustedPTS

最後に受け入れたPTS

→

endSession

範囲を閉じる

→

Receiver.finish

video / audio

→

finishWriting

最後のDataを出力

```
1 if lastAdjustedPTS.isValid {  
2     writer.endSession(atSourceTime: lastAdjustedPTS)  
3 }
```

Captureを止めてから、 両Writerの生成を完了させる

① Capture停止

新しい入力を止める



② writingQueue

投入済みcallbackを処理



③ HLS Writerをfinish

最後のDataを受信

fragment streamを閉じる



④ 保存用Writerをfinish

MP4の完成を確認

URLまたは生成エラーを返す

Sampleの現在の呼び出し順。HTTPの送信Taskはこの間も継続する

写真保存は、HTTP送信の完了を待たずに始める

時間 →	録画中	停止操作	終了処理
HLS送信	生成済みsegmentを順に送信		ENDLIST公開
端末MP4	撮影中から生成	MP4完成	
写真保存			写真へ追加

HLSの結果	写真保存の結果
公開完了 / 送信エラー	保存成功 / 完成MP4を保持
	保存失敗時は再試行

2つの結果を別々に確認。完了順は通信・保存の状況によって変わる

4つの運用制御を、配信の担当箇所へ配置



MomentNowで選んだ配置。SampleはiOS側でplaylistと公開順を管理

— FIT, NOT UNIVERSAL

この構成が効くのは、用途を絞ったとき

GOOD FIT

少人数・短時間・単一品質

新しいiPhone / 安定した上り回線



USE A MEDIA PLATFORM

大規模・長時間

複数画質の自動切替

厳しい可用性要件にも対応したい

「端末でできる」と「端末でやるべき」は別の判断

— TAKEAWAYS

端末内で生成し、保存できた範囲を公開する

生成

Camera /
Mic
からWriterへ
撮影データ
を入力



区切り

約2秒の
fMP4 Data
IDRと境界
を揃える



公開

保存成功後に
playlist更新
ViewerがGET



終了

最後の送信と
ENDLIST
保存MP4
も完了確認

AVCaptureSession → DataOutput → Receiver → AVAssetWriterDelegate

ありがとうございました

サンプルアプリのコード

<https://github.com/HikaruSato/iosdc2026HLSSample>

— APPENDIX

補足：Writerの時刻処理

開始の基準 / 10秒の理由 / 時刻を変えるコード

最初の映像を基準にWriterを開始

先に届いた音声

まだappendしない

最初の映像

`writer.start()`

10秒を開始位置に

映像・音声に共通

```
1 guard output === videoOutput else { return }
2 guard !didStartSession, let writer else { return }
3 try writer.start()
4 let pts = CMSampleBufferGetPresentationTimeStamp(sampleBuffer)
5 writer.startSession(atSourceTime: startTimeOffset)
6 timeOffsetDelta = startTimeOffset - pts
```

10秒の開始位置は、音声の時刻を前へ動かす余白

時間 → 0秒

映像の開始位置：10秒

映像

映像本体

AAC音声

priming

音声本体

priming分だけ音声の時刻を前へ補償しても、0秒より前にならない

```
1 private let startTimeOffset = CMTime(value: 10, timescale: 1)
2 writer.initialSegmentStartTime = startTimeOffset
```

模式図。10秒はAppleの設定例であり、再生開始まで待つ時間ではない

最初の映像で移動量を決め、 すべてのsampleに適用

移動量を最初に1回だけ決定

$$\text{delta} = 10s - \text{firstVideoPTS}$$

映像の全sample

$$\text{sourcePTS} + \text{delta}$$

音声の全sample

$$\text{sourcePTS} + \text{delta}$$

映像 100.00 → 10.00

音声 100.02 → 10.02

同じ-90秒。元の0.02秒の時間差を保つ

映像と音声それぞれの先頭を、別々に10秒へ合わせない

コピーの内容はそのまま、PTSと有効なDTSを補正

元のbuffer

映像・音声 + 元の時刻



独立したコピー

同じ内容 + 補正した時刻

```
1 var t = info
2 t.presentationTimeStamp = t.presentationTimeStamp + offset
3 if info.decodeTimeStamp.isValid {
4     t.decodeTimeStamp = t.decodeTimeStamp + offset
5 }
6 return t
```

timing infoを変換するmap内部の抜粋。新しいtimingでbufferをコピーする